1) Explain Kubernetes Cluster Architecture

Answer:
Kubernetes architecture is broadly divided into two parts: the **control plane** and the **data plane**. The control plane contains components like the **API server, etcd, scheduler, and controller manager**. The data plane contains **kubelet, container runtime, and kube-proxy** on the worker nodes.

When a request is sent using `kubectl`, it first reaches the **API server**, which validates the request through authentication and authorization. If the request is valid, the **scheduler** decides which worker node is best for the pod based on things like node affinity, tolerations, and available resources. Then the API server communicates with the **kubelet** on that node, and kubelet uses the **container runtime** to run the container. The created state is stored in **etcd**, and controllers such as the **replica set controller** are managed by the **controller manager**.

---

2) How various components of Kubernetes interact when you run `kubectl apply` (Pod)

Answer:
When you run `kubectl apply` to create a pod, the request first goes to the **API server**. The API server checks authentication and authorization to make sure the user has permission to create the resource. If the request is valid, it passes the request to the **scheduler**.

The scheduler chooses the right node for the pod based on node affinity, tolerations, taints, and available resources. After that, the API server talks to the **kubelet** on the selected node. Kubelet uses the **container runtime** to start the container inside the pod. The pod information is stored in **etcd**, and if the pod is managed by a controller, the **controller manager** ensures the desired state is maintained.

---

3) What is the purpose of services in Kubernetes?

Answer:
The main purpose of a Kubernetes Service is **service discovery**. Pods are ephemeral, so their IP addresses can change whenever they are recreated. Because of that, one pod should not communicate directly with another pod using a hardcoded IP address.

Instead, a pod communicates with a **Service**, and the Service forwards traffic to the correct pod using **labels and selectors**. A Service acts like a middleman between pods, ensuring communication continues even if the pod IP changes. Services can also provide **load balancing** across multiple pod replicas.

---

4) Why is hardcoding Pod IP communication a bad practice?

Answer:
Hardcoding a pod IP is a bad practice because pods are **ephemeral**. A pod can go down due to crashes, resource issues, or restart events, and when it comes back, it may get a **different IP address**. If another pod is hardcoded to use the old IP, communication fails and can lead to errors like **502 Bad Gateway**.

To avoid this, Kubernetes uses **Services**. Services do not rely on pod IPs; instead, they use **labels and selectors** to identify the right pod. This makes communication stable, flexible, and reliable even when pods are recreated.

---

5) What are the types of services in Kubernetes?

Answer:
The main types of Kubernetes Services are **ClusterIP, NodePort, LoadBalancer, ExternalName, and Headless Service**.

- **ClusterIP** is the default type and is accessible only **inside the cluster**. Any pod in the cluster can reach it using the service IP or DNS name.
- **NodePort** exposes the service on a fixed port on every node, so it can be accessed using **node IP + node port**.
- **LoadBalancer** creates an external load balancer and makes the service accessible **outside the cluster**, usually through a public IP.
- **ExternalName** maps a service to an external DNS name.
- **Headless Service** is created by setting `clusterIP: None`, and it is commonly used for **stateful apps** and **DNS-based service discovery**.

---

6) What are labels and selectors in Kubernetes?

Answer:
Labels are key-value pairs attached to Kubernetes resources to identify and group them. For example, a pod may have a label like `app=myapp`. Labels help organize resources and make them easier to query.

Selectors are used to find resources based on labels. Services and ReplicaSets use selectors to identify which pods they should target. For example, a Service uses a selector to route traffic to the correct pod, instead of using pod IP addresses directly.

---

7) What would you recommend: NodePort service or LoadBalancer service, and why?

Answer:
It depends on the access scope you need.

- Use **NodePort** if you want the service to be reachable within your **VPC** or through the node IP and port.
- Use **LoadBalancer** if you want the service to be accessible **externally/publicly** from outside the cluster.

If you want access only inside the cluster, **ClusterIP** is the better option. Also, LoadBalancer works properly only on Kubernetes clusters that have a **Cloud Controller Manager** or cloud provider integration. On clusters without it, LoadBalancer may not provide external access.

---

8) How are Kubernetes Services related to Kube Proxy?

Answer:
Services and **kube-proxy** work together to route traffic to pods. A Service provides stable access to pods, but it does not do the actual packet forwarding by itself. Instead, Kubernetes creates **Endpoints** for the Service, and **kube-proxy** watches those endpoints.

Kube-proxy updates **IP tables** on the worker nodes so that traffic sent to the Service IP is forwarded to the correct pod replicas. In simple terms, the Service gives a stable virtual entry point, and kube-proxy makes sure traffic reaches the matching pods behind it.

---

9) What is the disadvantage of LoadBalancer service type?

Answer:
The biggest disadvantage of a LoadBalancer Service is that it usually creates a **separate external load balancer for each service**. That can become **expensive** very quickly if you have many services exposed to the internet.

Another disadvantage is that it depends on the cloud provider and the **Cloud Controller Manager**. So in environments without cloud integration, it may not work as expected. For larger setups, **Ingress** is often a better solution because it can route traffic to multiple services using a single external load balancer.

---

10) What is a Headless Service in Kubernetes and when do you use it?

Answer:
A **Headless Service** is a Kubernetes Service created with `clusterIP: None`. Unlike a normal Service, it does **not perform load balancing**. Instead, it creates **DNS A records for each pod**, so clients can directly connect to individual pods.

You use it mainly for **stateful applications**, especially databases and StatefulSets, where each pod needs a stable identity and direct communication. For example, if you have multiple database replicas, a headless service lets the application reach each replica individually instead of routing all requests through one shared service IP.

If you want, I can also turn these into a **ready-to-submit interview notes format** or **short answers only**.

11) Can a Pod access a Service in a different namespace? If yes, how?

Answer:

Yes, a pod can access a Service in a different namespace. By default, Kubernetes Services of type **ClusterIP** are accessible throughout the cluster unless restricted by Network Policies.

The recommended way is to access the Service using its **DNS name** rather than its ClusterIP.

The DNS format is:

<service-name>.<namespace>.svc.cluster.local

For example, if a service named `payment-service` exists in the `payments` namespace:

payment-service.payments.svc.cluster.local

A pod in any namespace can use this DNS name to communicate with the service.

Although it is possible to access the Service using its ClusterIP, using the DNS name is considered the best practice because it is more reliable and easier to manage.

---

12) Explain how you can restrict access to a DB pod to only one application in the namespace.

Answer:

This can be achieved using **Kubernetes Network Policies**.

Suppose there are multiple application pods in a namespace and only one application pod should be allowed to access the MySQL database pod. By default, all pods within a namespace can communicate with each other, so additional restrictions are required.

The approach is:

1. Label the database pod.

```
labels:
  app: db
```

2. Label the application pod that should be allowed to access the database.

```
labels:
  app: payment-app
```

3. Create a Network Policy that allows ingress traffic to the database pod only from the authorized application pod.

Example:

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-payment-app
spec:
  podSelector:
    matchLabels:
      app: db
  ingress:
  - from:
    - podSelector:
        matchLabels:
          app: payment-app
```

With this configuration:

- `payment-app` can access the database.
- Other pods in the namespace cannot access the database.
- Access is controlled at the network level.

This is a common security practice for protecting sensitive database workloads.

---

13) Explain the deployment strategy that you follow in your organization.

Answer:

In our organization, we primarily follow the **Canary Deployment Strategy** for production releases.

The reason is that even after testing in development and staging environments, there is always a possibility that some issues may reach production. Deploying a new version directly to all users can create significant business impact.

In Canary Deployment:

1. The new version is initially released to a small percentage of users, typically around **10%**.
2. The application is monitored for issues, performance problems, and user feedback.
3. If everything works correctly, the rollout is gradually increased to **20%, 50%, and eventually 100%**.

4. If any issue is detected, the rollout can be stopped before impacting all users.

This approach minimizes risk and allows teams to identify problems early.

I am also familiar with **Blue-Green Deployment**, but Canary Deployment is the primary strategy used because it provides controlled and gradual rollout of changes.

---

14) Explain the rollback strategy that you follow in your organization.

Answer:

In our organization, we first try to avoid rollbacks by following a **Canary Deployment Strategy**.

When a new version is deployed:

- Initially, only a small percentage of users receive the new version.
- The rollout is gradually increased from 10% to 20%, then 50%, and finally 100%.
- This helps identify issues before they affect all users.

However, if a critical issue still reaches production, we follow a rollback process based on **GitOps**.

Our Kubernetes manifests, Helm charts, ConfigMaps, and Secrets are stored in Git repositories.

The rollback process is:

1. Revert the Helm chart or Kubernetes manifest to the previous stable version in Git.
2. Commit the change to the repository.
3. **Argo CD** continuously watches the Git repository.
4. Since auto-sync is enabled, Argo CD automatically detects the reverted version.
5. Argo CD synchronizes the Kubernetes cluster with the previous stable configuration.

This allows us to quickly restore the last known working version with minimal manual intervention.

---

15) Design a solution to avoid rollbacks.

Answer:

To avoid rollbacks as much as possible, I would implement a **Canary Deployment Strategy**.

The idea is to release the new application version gradually instead of exposing it to all users immediately.

The process would be:

1. Deploy the new version to only **10% of users**.
2. Monitor application performance, logs, metrics, and customer feedback.
3. Run additional validation and load tests.
4. If no issues are found, increase traffic gradually to:
    - 20%
    - 50%
    - 100%
5. Continue monitoring throughout the rollout process.

Benefits:

- Issues are detected early.
- Only a small subset of users is affected if problems occur.
- Production risk is significantly reduced.
- Rollbacks become much less frequent.

This approach is widely used by large organizations such as Amazon and Uber because it provides safer production deployments.

Even with this strategy, rollbacks may occasionally be required for unexpected issues or critical vulnerabilities, but the chances are greatly reduced.

---

16) Explain the deployment strategies that you used in the past.

Answer:

I have primarily worked with **Canary Deployment** and **Blue-Green Deployment** strategies.

Blue-Green Deployment

In Blue-Green Deployment:

- The current application version (Blue) runs in one environment.
- A completely separate environment (Green) is created for the new version.
- The new version is deployed to the Green environment.
- Once testing is completed successfully, the load balancer is switched from Blue to Green.

Example:

- V1 is running on existing servers.
- V2 is deployed on new servers with the same configuration.
- Traffic is redirected to V2 through the load balancer.

If any issue is discovered, traffic can immediately be redirected back to V1.

Advantages:

- Fast rollback.
- Minimal downtime.
- Safe production releases.

Canary Deployment

In Canary Deployment:

- The new version is first released to a small percentage of users.
- Traffic is gradually increased based on monitoring and validation results.

Example:

- 10% users receive V2.
- Then 20%.
- Then 50%.
- Finally 100%.

If issues are detected at any stage, the rollout can be paused or reverted before impacting all users.

Advantages:

- Reduced deployment risk.
- Early issue detection.
- Better user experience during releases.

---

17) Explain the role of CoreDNS in Kubernetes.

Answer:

CoreDNS is the default **DNS server** in Kubernetes.

Its primary role is to translate Kubernetes Service names into IP addresses, enabling service discovery inside the cluster.

For example, if a pod wants to communicate with a service:

payment-service.default.svc.cluster.local

The pod uses this DNS name instead of a hardcoded IP address.

CoreDNS resolves this DNS name to the corresponding Service IP and allows communication between workloads.

Benefits of CoreDNS:

- Service discovery
- DNS resolution inside the cluster
- Eliminates hardcoded IP addresses
- Supports communication between pods and services

In simple terms, CoreDNS performs the same role inside Kubernetes that traditional DNS servers perform on the internet.

---

18) A DevOps engineer tainted a node as NoSchedule. Can you still schedule a pod?

Answer:

Yes, it is possible by using **Tolerations**.

A taint marks a node and prevents pods from being scheduled onto it.

Example:

kubectl taint nodes node1 app=maintenance:NoSchedule

This tells Kubernetes not to schedule new pods on that node.

However, if a pod has a matching **Toleration**, Kubernetes can still schedule it on the tainted node.

Example:

```
spec:
  tolerations:
  - key: "app"
    operator: "Equal"
    value: "maintenance"
    effect: "NoSchedule"
```

With this toleration configured, the pod can be scheduled onto the tainted node despite the NoSchedule taint.

This mechanism is commonly used for:

- Dedicated workloads
- Maintenance operations
- Control-plane nodes
- Special-purpose nodes

---

19) Pod is stuck in CrashLoopBackOff. What steps will you take?

Answer:

CrashLoopBackOff is not an actual error. It is a Kubernetes state indicating that the container is repeatedly crashing and restarting.

My troubleshooting steps would be:

Step 1: Check Pod Logs

kubectl logs <pod-name>

Review application logs to identify:

- Application exceptions
- Configuration issues
- Database connection failures
- Startup failures

Step 2: Describe the Pod

kubectl describe pod <pod-name>

This provides information about:

- Events
- Restart counts
- Scheduling issues
- Probe failures
- Resource problems

Step 3: Verify Liveness and Readiness Probes

Incorrect probe configurations often cause containers to restart repeatedly.

For example:

- Wrong endpoint path
- Wrong port number
- Insufficient startup delay

A failed liveness probe can continuously restart the container, leading to CrashLoopBackOff.

Step 4: Check Resource Limits

Verify:

- CPU limits
- Memory limits
- OOMKilled events

Insufficient memory frequently causes application crashes.

Step 5: Validate Configuration

Check:

- ConfigMaps
- Secrets
- Environment variables

- Database connectivity
- External service dependencies

By analyzing logs, events, probes, and resource utilization, the root cause can usually be identified quickly.

---

20) What is the difference between Liveness Probe and Readiness Probe?

Answer:

Liveness Probe

A **Liveness Probe** checks whether the application inside the container is alive.

If the probe fails:

- Kubernetes assumes the application is unhealthy.
- The container is automatically restarted.

Purpose:

- Detect hung or dead applications.
- Recover automatically through restarts.

Example:

```
livenessProbe:
  httpGet:
    path: /health
    port: 8080
```

Readiness Probe

A **Readiness Probe** checks whether the application is ready to receive traffic.

If the probe fails:

- The pod is removed from Service endpoints.
- No traffic is sent to the pod.
- The container is NOT restarted.

Purpose:

- Prevent traffic from reaching applications that are still starting or temporarily unavailable.

Example:

```
readinessProbe:
  httpGet:
    path: /ready
    port: 8080
```

Key Difference

Liveness Probe	Readiness Probe
Checks if application is alive	Checks if application is ready to serve requests
Failure causes container restart	Failure removes pod from service endpoints
Used for recovery	Used for traffic management
Kubernetes restarts the container	Kubernetes stops sending traffic to the pod

In short:
Liveness = Should Kubernetes restart the container?
Readiness = Should Kubernetes send traffic to the container?

21) Explain the difference between Ingress and LoadBalancer Service Type.

Answer:

Both **Ingress** and **LoadBalancer Service Type** are used to expose applications to the external world and allow public access to Kubernetes workloads.

However, there are significant differences between them.

LoadBalancer Service Type

When you create a Service of type **LoadBalancer**, Kubernetes creates a dedicated load balancer for that particular service.

For example, on AWS:

- Service A → Load Balancer A
- Service B → Load Balancer B
- Service C → Load Balancer C

This means:

- Every exposed service gets its own load balancer.
- Costs increase when multiple services need public access.
- The cloud provider's Cloud Controller Manager decides which load balancer is created.
- You have limited control over the load balancer configuration.

Ingress

Ingress also exposes applications externally, but it works differently.

With Ingress:

- A single load balancer can be shared across multiple services.
- Traffic routing is configured using rules.
- Multiple applications can be exposed through the same load balancer.
- You get advanced routing capabilities.

Example:

example.com/app1 → Service 1
example.com/app2 → Service 2
api.example.com → Service 3

Advantages of Ingress

- Cost-effective
- One load balancer for multiple services
- Path-based routing
- Host-based routing
- SSL/TLS termination
- More configuration flexibility

Key Difference

LoadBalancer	Ingress

One load balancer per service	One load balancer for multiple services
Expensive for many services	More cost-effective
Limited routing options	Advanced routing features
Managed by Cloud Controller Manager	Managed by Ingress Controller

---

22) Your application works with ClusterIP but fails with Ingress. How do you troubleshoot it?

Answer:

If the application works through **ClusterIP** but fails through **Ingress**, it means:

- Pod is running.
- Service is working.
- Internal connectivity is functioning correctly.
- The issue most likely exists in the Ingress layer.

Step 1: Verify Ingress Resource

Check whether the Ingress resource exists:

kubectl get ingress

Verify:

- Hostname
- Paths
- Backend service name
- Backend service port

Step 2: Verify Ingress Controller

Check whether an Ingress Controller is installed and running:

kubectl get pods -n ingress-nginx

or

kubectl get pods -A

Without an Ingress Controller, Ingress resources do nothing.

Step 3: Verify Service Mapping

Ensure the Ingress points to the correct service.

Example:

```
backend:
  service:
    name: app-service
    port:
      number: 80
```

Check:

- Service name
- Service port

Step 4: Verify DNS Resolution

If a hostname is configured:

example.com

Verify that DNS resolves to the Ingress Controller's external IP.

Use:

nslookup example.com

or

dig example.com

Step 5: Check Ingress Controller Logs

kubectl logs <ingress-controller-pod>

Look for:

- Routing errors
- Configuration issues
- SSL issues
- Backend connectivity errors

Step 6: Validate Backend Service

Ensure the service has healthy endpoints:

kubectl get endpoints

If no endpoints exist, traffic cannot reach pods.

Step 7: Check TLS Configuration

If HTTPS is configured:

- Verify certificates.
- Verify TLS secret exists.
- Verify certificate validity.

Summary

Since ClusterIP already works, troubleshooting should focus on:

- Ingress resource configuration
- Ingress Controller
- DNS
- TLS
- Service mapping

---

23) Why do I need to set up an Ingress Controller after creating an Ingress?

Answer:

This is a fundamental Kubernetes question.

Creating an **Ingress Resource** alone does not expose an application.

An Ingress resource is simply a Kubernetes object.

Example:

kind: Ingress

Just like:

kind: Deployment

creates a Deployment object.

The difference is that Deployments already have controllers watching them.

For example:

- Deployment → ReplicaSet Controller watches it.
- ReplicaSet Controller creates Pods.

However, for Ingress:

- Kubernetes does not provide an Ingress Controller by default.
- No component automatically watches Ingress resources.

Therefore, an **Ingress Controller** must be installed.

The Ingress Controller:

1. Watches Ingress resources.
2. Reads routing rules.
3. Configures a load balancer or reverse proxy.
4. Routes traffic to backend services.

Without an Ingress Controller:

- Ingress resource exists.
- No traffic routing happens.
- Applications remain inaccessible externally.

Common Ingress Controllers:

- NGINX Ingress Controller
- HAProxy Ingress Controller
- Traefik
- AWS Load Balancer Controller
- Istio Gateway

Simple Explanation

Ingress is only a configuration object.
Ingress Controller is the component that implements that configuration.

---

24) We have an in-house load balancer. Can we use Ingress with our load balancer?

Answer:

Yes, absolutely.

Ingress is simply a Kubernetes resource that defines routing rules.

The actual implementation is performed by the **Ingress Controller**.

If your organization has its own proprietary load balancer, you can still use Ingress by developing or configuring a custom Ingress Controller.

How It Works

When an Ingress resource is created:

kind: Ingress

Nothing happens automatically.

An Ingress Controller must watch the Ingress resource and perform the necessary actions.

For example:

- NGINX Ingress Controller configures NGINX.
- AWS Load Balancer Controller configures AWS Load Balancers.
- A custom Ingress Controller can configure your in-house load balancer.

Architecture

Ingress Resource
↓
Custom Ingress Controller
↓
In-House Load Balancer
↓
Kubernetes Services
↓
Pods

Conclusion

Yes, an organization can integrate its proprietary load balancer with Kubernetes by implementing a custom Ingress Controller that watches Ingress resources and configures the load balancer accordingly.

---

25) Your Deployment has 3 replicas, but only 1 pod is running. What could be wrong?

Answer:

This is a common real-world troubleshooting question.

Since one pod is already running successfully, we can eliminate many common issues such as:

- Image not found
- Image pull secret issues
- Invalid image
- Basic application startup failures

The most likely issue is related to **scheduling constraints**.

Possible Cause 1: Insufficient Resources

The cluster may not have enough CPU or memory to schedule additional pods.

Check:

kubectl describe pod <pod-name>

Look for:

Insufficient CPU
Insufficient Memory

Possible Cause 2: Node Affinity

The deployment may require pods to run only on specific nodes.

Example:

nodeAffinity:

If only one matching node exists and resources are exhausted, remaining pods stay pending.

Possible Cause 3: Taints and Tolerations

Nodes may be tainted.

Example:

NoSchedule

If pods do not have matching tolerations, Kubernetes cannot schedule them.

Possible Cause 4: Pod Anti-Affinity

A deployment may intentionally prevent multiple replicas from running on the same node.

Example:

podAntiAffinity:

If enough nodes are unavailable, additional replicas remain pending.

Possible Cause 5: Resource Quotas

Namespace resource quotas may prevent creation of additional pods.

Check:

kubectl get resourcequota

Troubleshooting Commands

kubectl get pods
kubectl describe pod <pod-name>
kubectl get nodes
kubectl describe node <node-name>

The most likely explanation is usually **resource shortage or scheduling restrictions**.

---

26) Your pod mounts a ConfigMap, but changes to the ConfigMap are not reflected. Why?

Answer:

This depends on how the ConfigMap is consumed.

Assume an application reads a database hostname from a ConfigMap.

Initially:

db_host: my-db-server

Later it is updated:

db_host: new-db-server

But the application still uses the old value.

Possible Cause 1: Environment Variables

If the ConfigMap is consumed as environment variables:

envFrom:

or

env:

the values are loaded only during container startup.

Updating the ConfigMap does not automatically update running containers.

A pod restart is required.

Possible Cause 2: Application Caching

Even when ConfigMaps are mounted as volumes, the application may cache configuration values and never reread them.

The application itself may require a restart.

Possible Cause 3: ConfigMap Mounted as Volume

When mounted as a volume:

volumeMounts:

Kubernetes usually updates the mounted files automatically after some time.

However:

- There may be a delay.
- The application must reread the file.

Solution

If ConfigMap is consumed through environment variables:

kubectl rollout restart deployment <deployment-name>

This recreates pods and reloads updated values.

Key Point

ConfigMap updates do not automatically refresh environment variables inside running containers.

---

27) Explain how Node Affinity works and when you use it.

Answer:

Node Affinity is a Kubernetes feature used to influence pod scheduling.

It allows you to tell Kubernetes which nodes a pod should run on based on node labels.

Example:

Suppose a cluster contains:

- Node 1 → CPU
- Node 2 → CPU
- Node 3 → GPU

An AI/ML workload should run only on GPU nodes.

Node Affinity can enforce this requirement.

Types of Node Affinity

Required During Scheduling Ignored During Execution
requiredDuringSchedulingIgnoredDuringExecution

This is a hard requirement.

If no matching node exists:

- Pod remains Pending.
- Scheduling fails.

Preferred During Scheduling Ignored During Execution
preferredDuringSchedulingIgnoredDuringExecution

This is a soft requirement.

Kubernetes tries to schedule the pod on preferred nodes.

If unavailable:

- Pod can run on another node.

Common Use Cases

- GPU workloads
- Database workloads
- High-memory nodes
- Dedicated application nodes
- Compliance requirements

Summary

Node Affinity allows scheduling decisions based on node labels and provides both hard and soft scheduling preferences.

---

28) What is the difference between Node Affinity and Node Selector?

Answer:

Node Affinity is the modern and more flexible version of Node Selector.

Node Selector

Node Selector supports only simple key-value matching.

Example:

```
nodeSelector:
  diskType: SSD
```

Kubernetes schedules the pod only on nodes having:

diskType=SSD

Limitations:

- Simple matching only.
- No advanced operators.
- No soft preference support.

Node Affinity

Node Affinity provides:

- Hard affinity
- Soft affinity
- Multiple operators
- More flexibility

Example:

requiredDuringSchedulingIgnoredDuringExecution

Hard requirement.

Example:

preferredDuringSchedulingIgnoredDuringExecution

Soft preference.

Supported operators include:

- In
- NotIn
- Exists
- DoesNotExist
- Gt
- Lt

Comparison

Node Selector	Node Affinity
Simple key-value matching	Advanced matching
Hard requirement only	Hard and soft requirements
Limited flexibility	Highly flexible
Older approach	Modern approach

Summary

Node Affinity should generally be preferred because it provides significantly more control over pod scheduling.

---

29) What is Container Runtime in Kubernetes?

Answer:

Container Runtime is a component present on every worker node in a Kubernetes cluster.

Its responsibility is to run containers.

How It Works

When a user creates a pod:

1. Request reaches the API Server.
2. API Server performs authentication and authorization.
3. Scheduler selects an appropriate node.
4. API Server communicates with Kubelet.
5. Kubelet invokes the Container Runtime.
6. Container Runtime creates and runs containers.

Responsibilities of Container Runtime

Pull Container Images

Example:

Docker Hub
Artifact Registry

ECR
ACR

The runtime downloads the required image.

Create Linux Namespaces

Namespaces provide isolation for:

- Process IDs
- Network
- Mount points
- Hostnames

Configure Cgroups

Cgroups enforce:

- CPU limits
- Memory limits
- Resource isolation

Start Containers

The runtime launches the container process and manages its lifecycle.

Common Container Runtimes

- containerd
- CRI-O
- Docker Engine (historically via dockershim)

Summary

Container Runtime is the component responsible for pulling images, creating namespaces and cgroups, and ultimately running containers on Kubernetes worker nodes.